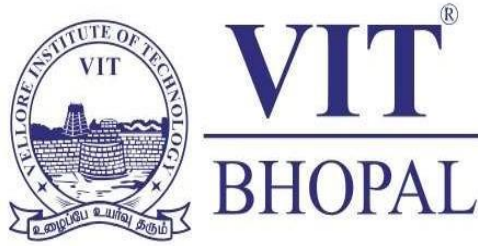


# VITYARTHI – PROJECT



**Project title: Aircraft Forces Calculator**

**Course title: Introduction to Problem Solving And  
Programming in Python**

**Course code: CSE1021**

**Course type: Flipped course**

**Course credits: 4**

**Professor: Dr. Sasmita Padhy**

**Slot: A11+A12+A13+A14+D11+D12**

**Submitted by: Shayam Mohan. M**

**Registration number: 26BAS10077**

**Submission date: 27/09/2026**

# VITYARTHI – PROJECT

## INTRODUCTION :

- The Airplane Four Forces Calculator is a computational tool designed to analyze the four fundamental forces acting on an aircraft during flight: Lift, Drag, Thrust, and Weight. Understanding these forces is essential in aerospace engineering for studying aircraft performance and flight conditions.
- The calculator uses parameters such as aircraft mass, wing area, velocity, altitude, lift coefficient, drag coefficient, and thrust to calculate the forces acting on the aircraft. It also considers atmospheric properties such as air temperature, pressure, and density for the calculations.
- The system calculates Lift and Drag using aerodynamic equations, Weight from the aircraft mass and gravitational acceleration, and Thrust from the engine input. It further determines the net force, acceleration, Lift-to-Drag ratio, and approximate flight condition of the aircraft.
- The project demonstrates the application of aerospace engineering concepts and programming to develop a simple and practical aircraft-performance calculator. It also provides graphical analysis of the four forces with respect to airspeed, helping students understand how changes in flight parameters affect aircraft performance.

## PROBLEM STATEMENT :

- Aircraft performance depends on the balance between four fundamental forces: Lift, Drag, Thrust, and Weight. Determining these forces for different flight conditions through manual calculations can be time-consuming and may result in calculation errors.
- The Airplane Four Forces Calculator is developed to provide a simple and efficient method for calculating these forces using parameters such as aircraft mass, wing area, velocity, altitude, lift

# VITYARTHI – PROJECT

coefficient, drag coefficient, and thrust. The system processes the given inputs and calculates the corresponding aerodynamic forces.

- The calculator also provides additional results such as net force, acceleration, and Lift-to-Drag (L/D) ratio, helping users understand the overall performance of the aircraft under the given flight conditions.
- The project aims to reduce repetitive manual calculations and provide an easy-to-use educational tool for understanding the relationship between aircraft parameters and the four fundamental forces of flight.

## PROJECT OBJECTIVES:

- **Calculate the Four Fundamental Forces:** To develop a calculator that determines the Lift, Drag, Thrust, and Weight acting on an aircraft under given flight conditions.
- **Apply Aerodynamic Principles:** To implement standard aerodynamic equations using parameters such as velocity, wing area, lift coefficient, drag coefficient, and air density.
- **Consider Atmospheric Conditions:** To calculate temperature, pressure, and air density based on altitude using an atmospheric model.
- **Analyze Aircraft Performance:** To determine additional parameters such as net force, acceleration, and Lift-to-Drag (L/D) ratio for the given flight condition.
- **Determine Flight Condition:** To compare Lift with Weight and Thrust with Drag to identify the approximate flight condition of the aircraft.
- **Provide Graphical Analysis:** To visualize how Lift, Drag, Thrust, Weight, and L/D ratio vary with airspeed, making aircraft performance easier to understand.

# VITYARTHI – PROJECT

- Create an Educational Tool: To combine programming and aerospace engineering concepts into a simple calculator that can assist students in understanding the basic principles of aircraft flight mechanics.

## FUNCTIONAL REQUIREMENTS :

### 1. User Input:

- The system shall accept important aircraft parameters such as mass, wing area, velocity, altitude, lift coefficient, drag coefficient, and thrust.
- The system shall validate the input values and prevent invalid or non-physical values from being used in calculations.

### 2. Atmospheric Calculation:

- The system shall calculate air temperature, pressure, and density based on the specified altitude.
- The atmospheric model shall support calculations for different flight altitudes.

### 3. Four Forces Calculation:

- The system shall calculate the four fundamental aircraft forces:
  - Lift
  - Drag
  - Weight
  - Thrust
- Lift and Drag shall be calculated using dynamic pressure, wing area, and their respective aerodynamic coefficients.

### 4. Performance Analysis:

- The system shall calculate net force, acceleration, and Lift-to-Drag (L/D) ratio.
- It shall determine the approximate flight condition by comparing

# VITYARTHI – PROJECT

Lift with Weight and Thrust with Drag.

## 5. Graphical Visualization:

- The system shall generate graphs showing Lift, Drag, Thrust, Weight, and L/D ratio against airspeed.
- A four-force comparison graph shall also be provided for visual analysis.

## 6. Result Display:

- The system shall display the calculated values in a clear and readable format.
- Results shall include atmospheric properties, four forces, net force, acceleration, L/D ratio, and flight condition.

## NON-FUNCTIONAL REQUIREMENTS :

The Airplane Four Forces Calculator should satisfy the following non-functional requirements:

### 1. Performance

The system should perform force and aircraft-performance calculations quickly without unnecessary processing delays.

### 2. Usability

The calculator should provide clear inputs and readable outputs so that students can easily understand the calculated aircraft parameters and forces.

### 3. Reliability

The system should validate important input values and prevent invalid calculations, such as using non-positive aircraft mass or wing area.

### 4. Accuracy

The calculations should use standard physical constants and appropriate aerodynamic equations to provide consistent results.

# VITYARTHI – PROJECT

## 5. Portability

The project should be executable on systems that have Python and the required libraries such as NumPy and Matplotlib installed.

## 6. Maintainability

The program should be organized into separate functions for atmospheric calculations, force calculations, flight-condition analysis, and graph generation, making future modifications easier.

## 7. Offline Operation

The calculator should perform its calculations and generate graphs without requiring an internet connection.

## SYSTEM ARCHITECTURE :

The Airplane Four Forces Calculator follows a modular architecture in which the user inputs aircraft and flight parameters, the system processes the data using aerodynamic and atmospheric calculations, and the results are displayed along with graphical analysis.

The main components of the system are:

### 1. Input Module

Accepts aircraft mass, wing area, velocity, altitude, lift coefficient, drag coefficient, and thrust as input parameters.

### 2. Atmospheric Model

Calculates air temperature, pressure, and density based on the given altitude using an atmospheric model.

### 3. Force Calculation Module

Uses the calculated air density and user inputs to determine Lift, Drag, Weight, and Thrust. It also calculates net force,

# VITYARTHI – PROJECT

acceleration, and the Lift-to-Drag ratio.

## 4. Flight Condition Module

Compares Lift with Weight and Thrust with Drag to identify the approximate flight condition, such as level flight, accelerating, decelerating, climbing, or descending.

## 5. Graph Generation Module

Generates graphs showing the relationship between airspeed and Lift/Weight, Drag/Thrust, and L/D ratio. It also provides a four-force comparison.

## 6. Output Module

Displays the atmospheric properties, calculated forces, performance parameters, and identified flight condition to the user.

# VITYARTHI – PROJECT

Overall Flow:

User Inputs



Input Validation



Atmospheric Calculation

(Temperature → Pressure → Air Density)



Force Calculation

(Lift + Drag + Weight + Thrust)



Performance Analysis

(Net Force + Acceleration + L/D Ratio)



Flight Condition

(Level / Accelerating / Decelerating / Climbing / Descending)



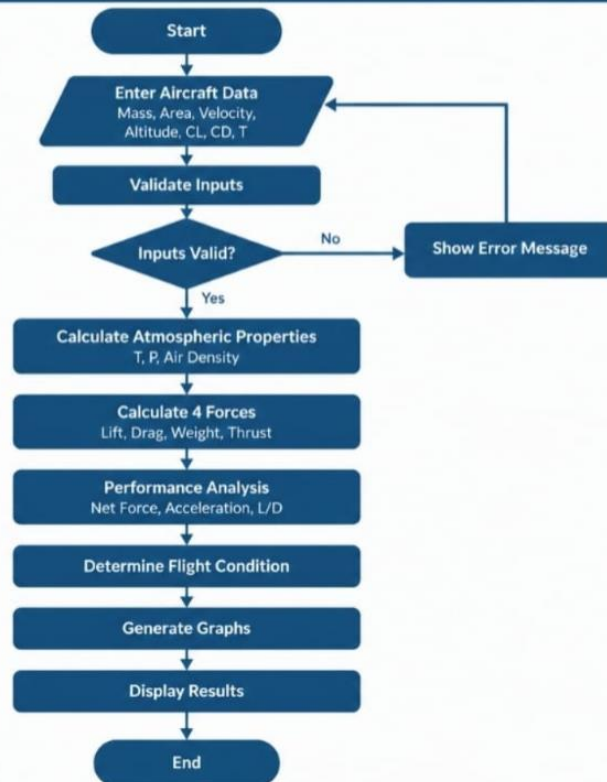
Graph Generation & Results

# VITYARTHI – PROJECT

## DESIGN DIAGRAMS :

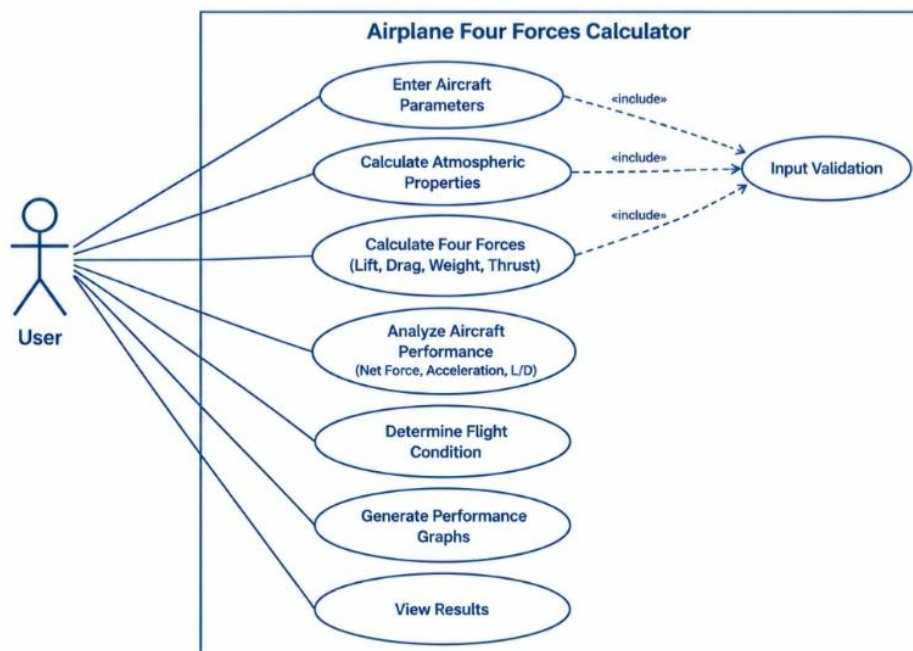
### 1.Workflow :

#### 1. Workflow Diagram



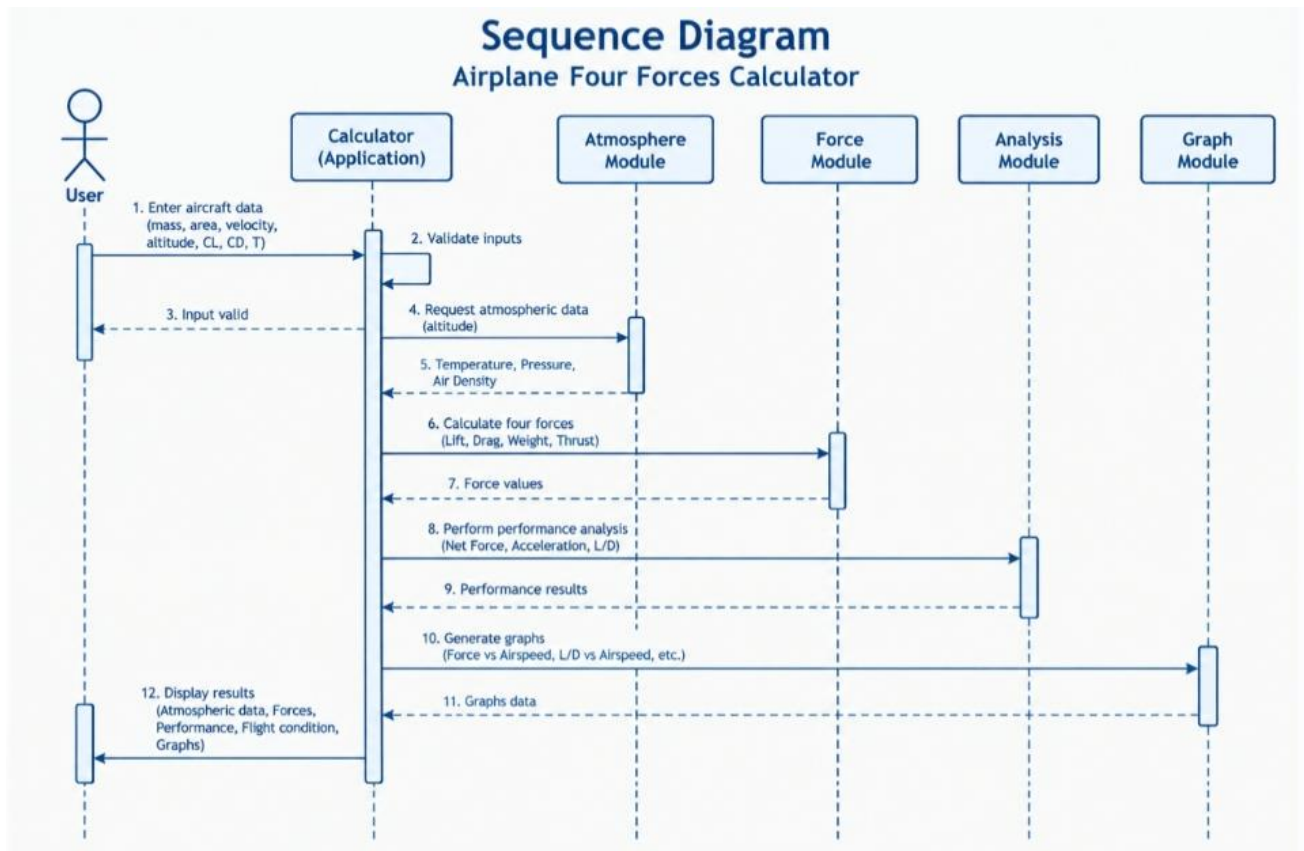
### 2.Use case Diagram :

#### Use-Case Diagram Airplane Four Forces Calculator

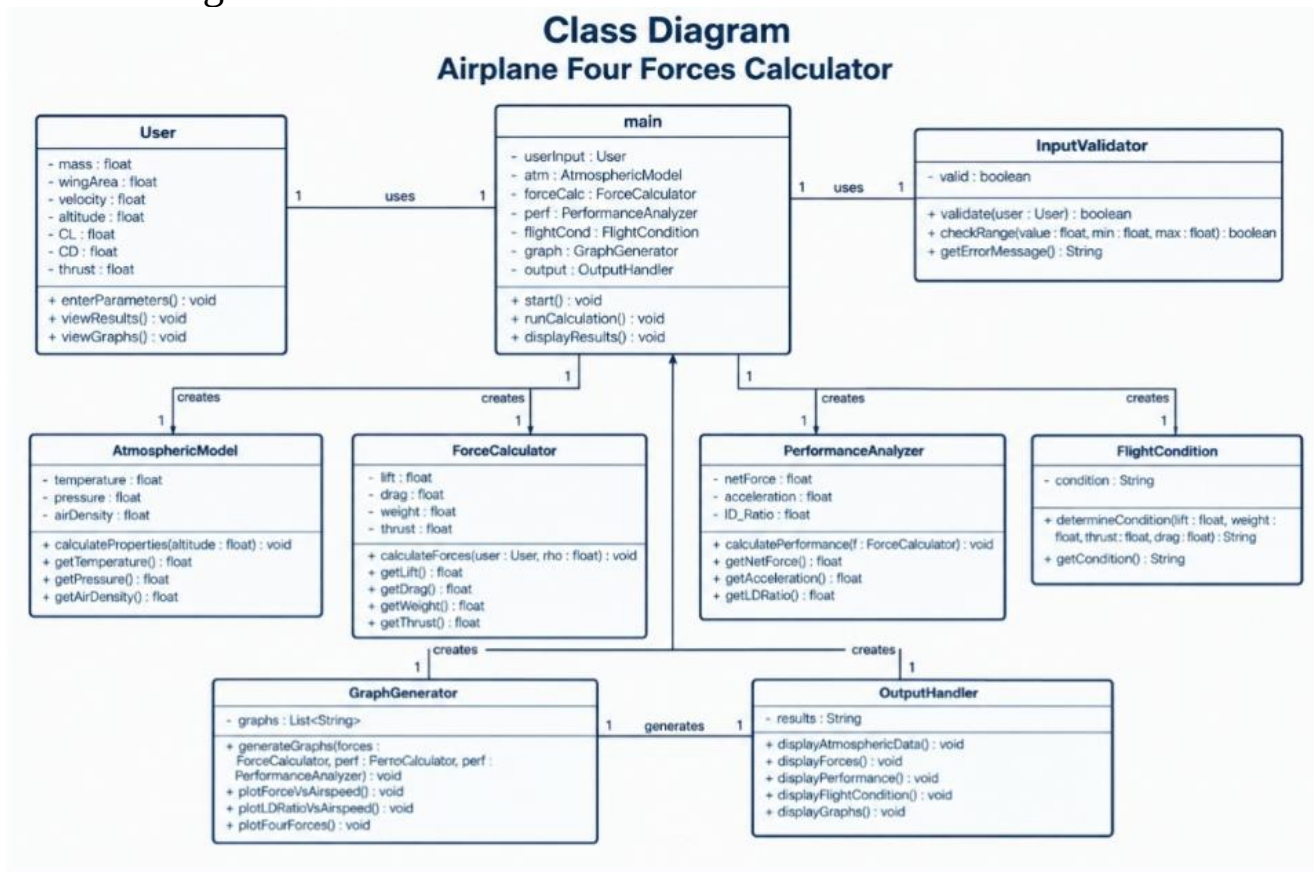


# VITYARTHI – PROJECT

## 3.Sequence Diagram :



## 4.Class Diagram :



# VITYARTHI – PROJECT

## 5.Data Flow Architecture

### 1. Input Stage:

Captures raw physical and operational parameters: aircraft mass, wing area, velocity, altitude, lift coefficient ( $C_L$ ), drag coefficient ( $C_D$ ), and engine thrust.

### 2. Validation & Atmospheric Properties Stage:

Validates input constraints (e.g., ensuring mass and wing area are strictly positive to prevent numerical errors like division by zero).

Processes the altitude input through the International Standard Atmosphere (ISA) model to compute corresponding ambient temperature, pressure, and air density.

### 3. Computation / Execution Stage:

Computes dynamic pressure using air density and velocity values.

Calculates fundamental aerodynamic and physical forces, including Lift, Drag, Weight, Vertical Force, Horizontal Force, Net Force, Acceleration, and the Lift-to-Drag (L/D) ratio.

### 4. Classification Stage:

Analyzes the balance between vertical and horizontal forces against defined tolerances to determine the current flight condition (e.g., Level Flight, Climbing, Descending, Accelerating, or Decelerating).

### 5. Presentation & Output Stage:

Formats numerical results to fixed decimal precision and prints organized performance logs to the console.

Generates and exports visual plots (such as force vs. airspeed curves and a four-forces bar chart) for comprehensive technical analysis.

# VITYARTHI – PROJECT

## 6.Component Breakdown

- Main Driver Module (main): Handles script entry, coordinates parameter configuration, and delegates execution between calculation, classification, and output routines.
- Input Validation Component: Ensures input parameters (such as mass and wing area) are valid and positive numbers to prevent runtime calculation exceptions.
- Atmospheric & Environment Engine (atmospheric\_properties): Processes altitude inputs using standard atmospheric physics to compute temperature, pressure, and air density values.
- Calculation Engine (calculate\_forces): Coordinates intermediate formulas, computes dynamic pressure, and derives aerodynamic forces, net force, acceleration, and the L/D ratio.
- Flight Condition Module (determine\_flight\_condition): Analyzes force balances and evaluates vertical/horizontal differences against defined tolerances to categorize the operational flight status.
- Output Renderer & Visualization Module: Formats and prints text-based performance logs to standard output, and generates structured graphical plots using Matplotlib.

## DESIGN DECISIONS & RATIONALE :

### 1. Programming Language: Python

Rationale: Python's numerical computing ecosystem (NumPy, Matplotlib) makes it well suited for scientific/engineering calculations and visualization, with far less boilerplate than a compiled language for this kind of exploratory analysis.

### 2. Atmospheric Model: International Standard Atmosphere (ISA), Piecewise

Rationale: Rather than a single formula, the model splits at 11 km — the troposphere uses the exact ISA lapse-rate equations, while above it a simplified exponential (constant-temperature) approximation is used.

# VITYARTHI – PROJECT

This balances physical accuracy where most flight occurs against implementation simplicity at altitudes the script isn't primarily designed for.

## 3. Modular Function Architecture

`atmospheric_properties`, `calculate_forces`, `determine_flight_condition`, and `generate_graphs` are kept as separate, single-purpose functions rather than one monolithic block.

Rationale: Each stage of the pipeline (environment → forces → classification → visualization) can be tested, reused, or swapped independently — e.g., `atmospheric_properties` is reused by both the force calculator and the graph generator without duplicating logic.

## 4. Dictionary-Based Results Structure

`calculate_forces` returns a single dictionary rather than a fixed tuple or a custom class.

Rationale: A dictionary allows the result set to be iterated generically for display (as the print loop does) without hardcoding each field name, and makes it easy to add or remove computed quantities later without breaking the function signature.

## 5. Tolerance-Based Flight Condition Classification

`determine_flight_condition` uses a 5% margin on weight and thrust rather than checking for exact equality.

Rationale: Real force balances are never exactly zero due to floating-point arithmetic and continuous physical variables; a tolerance band is necessary to meaningfully classify "LEVEL FLIGHT" instead of the condition never triggering.

## 6. Vector-Magnitude Approach for Net Force

Vertical and horizontal force imbalances are combined via

# VITYARTHI – PROJECT

$\sqrt{\text{vertical}^2 + \text{horizontal}^2}$  into one net force and one acceleration figure.

Rationale: Simplifies the two independent force imbalances (lift-vs-weight, thrust-vs-drag) into a single intuitive scalar for the results printout, at the cost of not preserving direction — acceptable here since direction is already conveyed separately by the flight-condition label.

## 7. Non-Interactive Plotting Fallback (Agg Backend)

The script detects whether a display is available and falls back to saving a PNG instead of calling `plt.show()`.

Rationale: Ensures the script runs reliably across environments (local machine with GUI vs. headless server/CI), rather than assuming a graphical backend is always present.

## 8. Defensive Input Validation

`calculate_forces` raises a `ValueError` for non-positive mass or `wing_area` before any computation runs.

Rationale: Prevents a silent `ZeroDivisionError` or physically meaningless results (infinite acceleration) from propagating downstream, surfacing the problem at its source instead.

## 9. In-Memory, Single-Run Execution Model

No database, file caching, or persistent state — each run computes fresh from the input parameters.

Rationale: The computation is lightweight and deterministic given its inputs, so there's no benefit to persistence; keeping everything in memory keeps the script simple and dependency-free.

# VITYARTHI – PROJECT

## IMPLEMENTATION DETAILS :

- Language: Python 3
- Development Environment: VS Code / Jupyter / any standard Python 3 interpreter with NumPy and Matplotlib installed
- Source File: aircraft\_corrected.py
- Key Libraries: math (scalar math functions), numpy (velocity array generation for plotting), matplotlib.pyplot (visualization)

### Core Functions:

1. `atmospheric_properties(altitude)` — Computes temperature, pressure, and air density at a given altitude using the ISA model (exact equations below 11 km, simplified exponential decay above it). Clamps negative altitude inputs to 0.
2. `calculate_forces(mass, wing_area, velocity, altitude, cl, cd, thrust)` — Validates mass and wing\_area are positive, calls `atmospheric_properties` to get air density, then computes dynamic pressure, lift, drag, weight, vertical/horizontal force imbalance, net force, acceleration, and lift-to-drag ratio. Returns all results as a dictionary.
3. `determine_flight_condition(lift, weight, thrust, drag)` — Applies a 5% tolerance band to the vertical (lift vs. weight) and horizontal (thrust vs. drag) force differences to classify the aircraft's state as LEVEL FLIGHT, CLIMBING, DESCENDING, ACCELERATING, or DECELERATING.
4. `generate_graphs(mass, wing_area, altitude, cl, cd, thrust, save_path=None)` — Sweeps velocity from 10–150 m/s using `numpy.linspace`, recomputing lift, drag, and L/D ratio at each point. Produces a 2×2 matplotlib figure:
  - Lift & Weight vs. Airspeed
  - Drag & Thrust vs. Airspeed
  - Lift-to-Drag Ratio vs. Airspeed
  - Four Fundamental Forces bar chart at a fixed 80 m/s reference speed

# VITYARTHI – PROJECT

Detects the active Matplotlib backend and either displays the figure interactively (`plt.show()`) or saves it to `save_path` when no display is available.

5. `if __name__ == "__main__":` block — Defines default input parameters (mass, wing area, velocity, altitude, `cl`, `cd`, thrust), runs the full calculation and classification pipeline, prints a formatted results table to the console (with a label override so `ld_ratio` displays as "L/D Ratio"), and calls `generate_graphs`.

6. Execution Flow:

Input parameters → `calculate_forces` (which internally calls `atmospheric_properties`) → `determine_flight_condition` → console output → `generate_graphs` for visualization.

## TESTING APPROACH : Aircraft Performance Script

1. Unit-Level Verification:

Validated `atmospheric_properties` against known ISA reference values (e.g., sea-level: 288.15 K, 101,325 Pa,  $\sim 1.225 \text{ kg/m}^3$ ) to confirm the temperature, pressure, and density equations are correctly implemented.

2. Execution/Runtime Testing:

Ran the full script end-to-end with the default input set (mass = 1200 kg, wing area =  $16 \text{ m}^2$ , velocity = 70 m/s, altitude = 1000 m) and confirmed it completes without exceptions, producing consistent numerical output across repeated runs.

3. Boundary & Edge-Case Testing:

- Tested mass = 0 and wing\_area = 0 to confirm the added validation raises a clear `ValueError` instead of crashing with a `ZeroDivisionError`.
- Tested altitude < 0 to confirm it is clamped to 0 rather than producing invalid atmospheric values.
- Tested drag = 0 (e.g., `cd` = 0) to confirm the L/D ratio

# VITYARTHI – PROJECT

safely defaults to 0 instead of raising a division error.

## 4. Flight-Condition Classification Testing:

Checked the tolerance-based logic in `determine_flight_condition` against multiple velocity values to confirm it correctly distinguishes CLIMBING, DESCENDING, and LEVEL FLIGHT cases, and verified the 5% tolerance band behaves sensibly near the lift-equals-weight crossover point (~40 m/s for the given `cl`).

## 5. Output Formatting Testing:

Confirmed the results printout displays all fields with two-decimal precision and that the `ld_ratio` label override correctly renders as "L/D Ratio" instead of the auto-generated "Ld Ratio".

## 6. Environment/Display Testing:

Ran the script in a headless (no-display) environment to confirm the Matplotlib backend detection correctly falls back to saving `aircraft_graphs.png` instead of silently failing on `plt.show()`.

## 7. Visual Inspection of Graphs:

Manually reviewed the generated 2×2 plot (Lift/Weight, Drag/Thrust, L/D ratio, and the four-forces bar chart at 80 m/s) to confirm the curves follow expected physical trends — e.g., lift and drag both increasing with the square of velocity, and L/D ratio remaining constant since it depends only on `cl/cd`, not velocity.

## CHALLENGES FACED :

### 1. Understanding Aerodynamic Calculations:

Applying the equations for Lift, Drag, Weight, and Thrust correctly required understanding the relationship between aircraft parameters and aerodynamic forces.

# VITYARTHI – PROJECT

## 2. Atmospheric Property Calculation:

Calculating temperature, pressure, and air density based on altitude was an important challenge because air density directly affects Lift and Drag calculations.

## 3. Input Validation:

The program needed to handle invalid values and prevent incorrect calculations, particularly for parameters such as aircraft mass and wing area.

## 4. Flight Condition Analysis:

Determining the approximate flight condition required comparing Lift with Weight and Thrust with Drag using suitable tolerance values.

## 5. Graphical Representation:

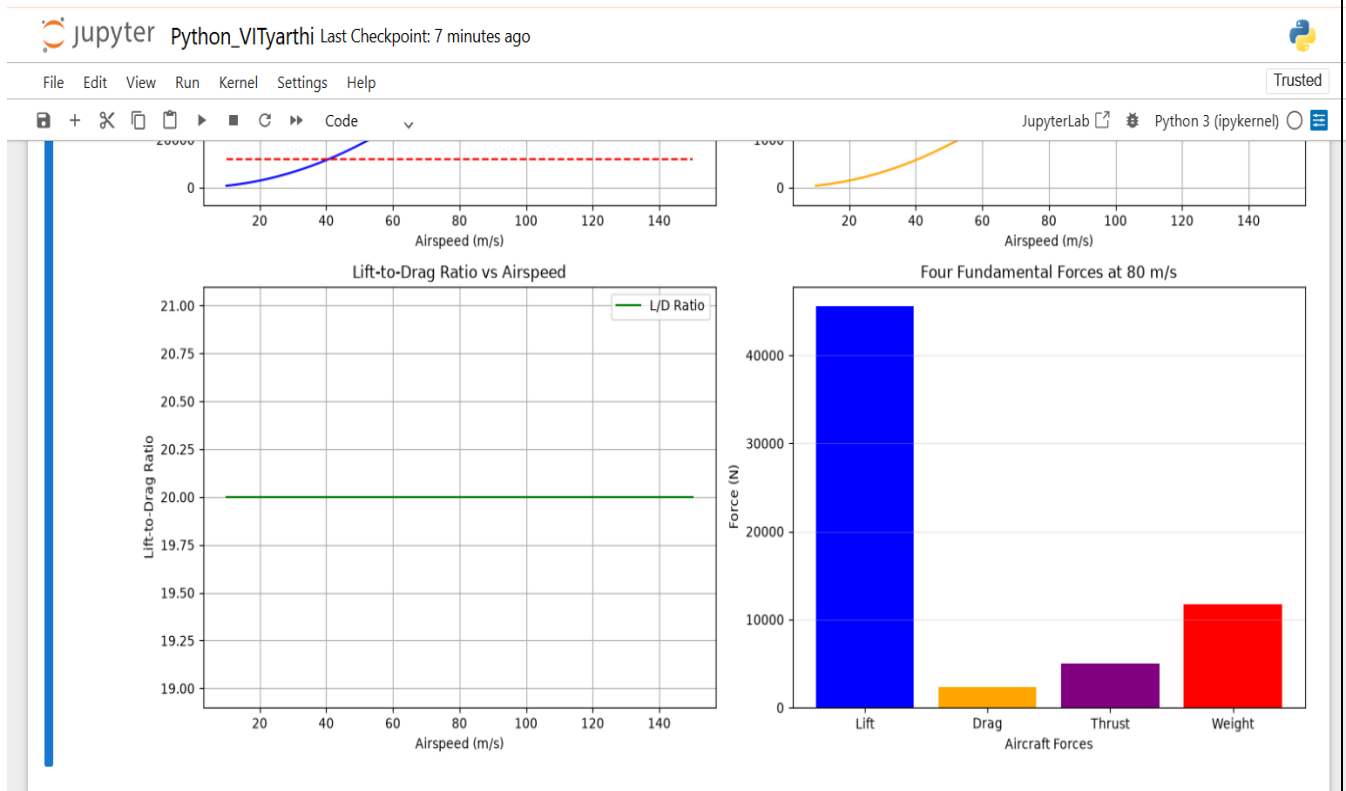
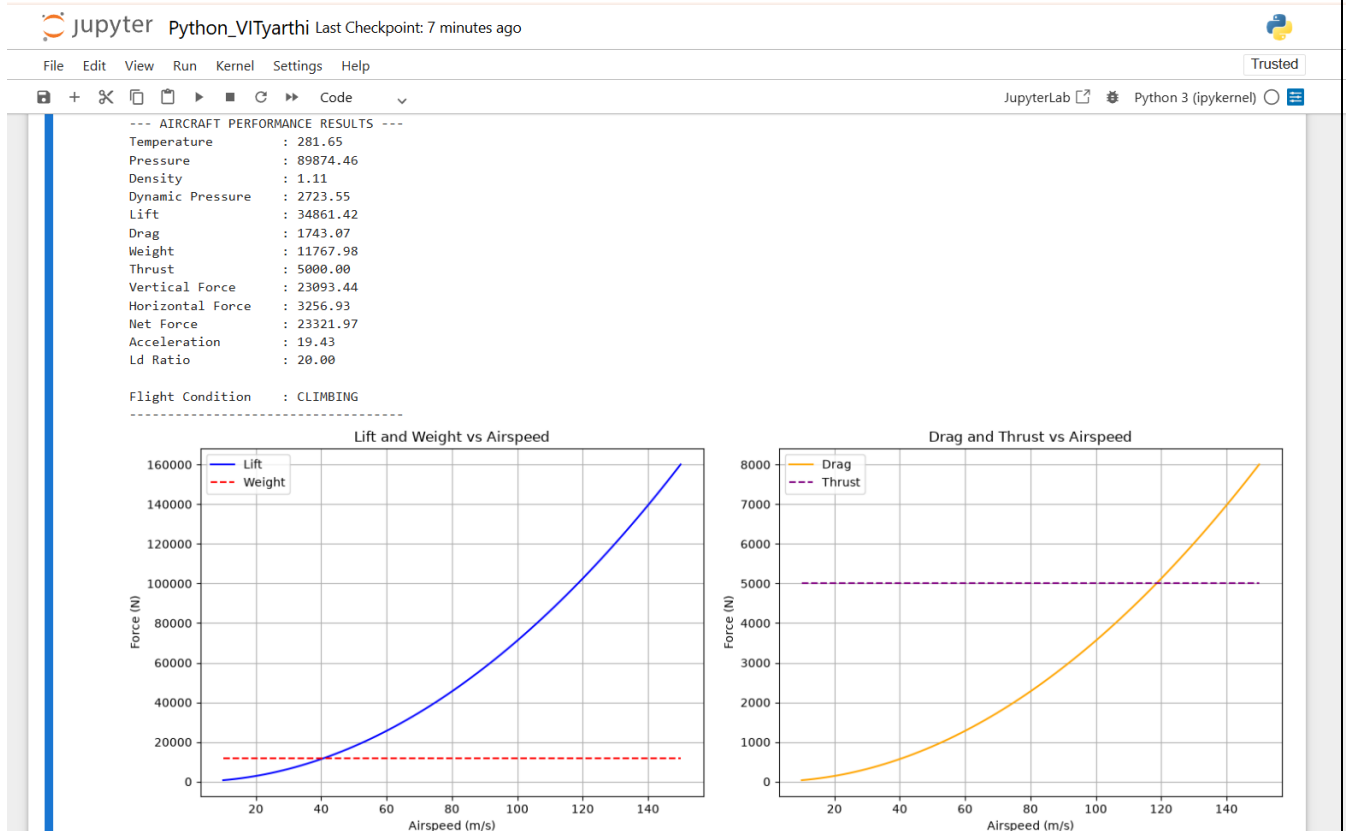
Generating meaningful graphs for Lift, Drag, Weight, Thrust, and L/D ratio required organizing the calculated data properly and presenting it clearly.

## 6. Integration of Multiple Modules:

Combining atmospheric calculations, force calculations, performance analysis, flight-condition detection, and graph generation into one working Python program required careful organization and testing.

# VITYARTHI – PROJECT

## OUTPUT :



# VITYARTHI – PROJECT

## LEARNINGS AND KEY TAKEAWAYS :

### 1. Understanding the Four Forces of Flight:

Gained a better understanding of Lift, Drag, Thrust, and Weight and their importance in aircraft flight.

### 2. Application of Aerodynamic Equations:

Learned how aircraft parameters such as velocity, wing area, air density, CL, and CD are used to calculate aerodynamic forces.

### 3. Atmospheric Modeling:

Learned how altitude affects temperature, pressure, and air density and how these properties influence aircraft performance.

### 4. Python Programming:

Improved programming skills by implementing calculations, functions, input validation, and conditional logic.

### 5. Data Visualization:

Learned to use graphs to represent aircraft performance and understand how forces change with airspeed.

### 6. Performance Analysis:

Learned how to calculate net force, acceleration, and Lift-to-Drag ratio and use them to analyze flight conditions.

### 7. Problem-Solving Skills:

Improved the ability to divide a technical problem into smaller modules and integrate them into a complete working application.

# **VITYARTHI – PROJECT**

## **FUTURE ENHANCEMENTS :**

### **1. Graphical User Interface (GUI):**

Develop a user-friendly interface where users can enter aircraft parameters and view results interactively.

### **2. Real-Time Flight Simulation**

Add a simulation feature to show how the four forces change continuously during flight.

### **3. Advanced Atmospheric Models**

Include more detailed atmospheric models for different altitudes and flight conditions.

### **4. More Aircraft Parameters**

Add parameters such as angle of attack, wing loading, aspect ratio, and aircraft configuration for more detailed analysis.

### **5. Interactive Graphs**

Allow users to change input values and observe the corresponding changes in performance graphs.

### **6. Aircraft Comparison**

Add a feature to compare the performance of two or more aircraft under similar flight conditions.

### **7. Data Export**

Provide options to save calculated results and graphs in formats such as CSV or PDF.

# VITYARTHI – PROJECT

## 8. Advanced Flight Analysis

Extend the project to estimate parameters such as stall speed, maximum L/D condition, climb performance, and range.

These enhancements can make the project more interactive and useful for studying aircraft performance and aerospace engineering concepts.

## REFERENCES :

- **NASA Glenn Research Center** – Beginner's Guide to Aerodynamics: The Four Forces of Flight.
- **Python Documentation** – Python Language Reference and Standard Library Documentation.
- **NumPy Documentation** – Array Operations and Numerical Computing.
- **Matplotlib Documentation** – Plotting and Data Visualization in Python.
- **International Standard Atmosphere (ISA)** – Standard Atmospheric Properties and Aircraft Performance Calculations.

# VITYARTHI – PROJECT